DAB111 Python Cheat Sheet

Weeks 1 to 3 and notebook essentials

Source summaries and adapted examples. * = added clarification or study reminder. Source key below.

Week 1 Introduction to Programming

Programming: instructions to receive, process, store and send data. **Data** = raw facts; **information** = processed results. **Hardware** = equipment; **software** = programs.

Bit / nibble / byte: binary digit / 4 bits / 8 bits. Decimal = base 10; binary = base 2; hex = base 16 (0-9, a-f). One hex digit = 4 bits.

CPU: Control Unit fetches, decodes, executes, stores; ALU calculates and compares. **RAM** is volatile; **ROM** retains startup instructions. Python uses an **interpreter**.

Algorithm: correct, efficient, understandable steps. **Sequence structure:** execute in order. Develop: understand → plan → write → test → deploy → maintain. Plan **input** → **processing** → **output** using pseudocode or flowcharts; desk-check on paper.

Flowchart: oval start/stop; rectangle process; parallelogram I/O; diamond decision. **Errors:** syntax breaks rules; runtime occurs during execution; logic gives wrong results. Test units, integration and whole system.

Variables types and operators

Assignment: `x = 5`. Objects have value, type and identity: `type(x)`, `id(x)`. Types: `int, float, str, bool`. Cast with `int(), float(), str(), bool()`. Names are case-sensitive; use descriptive `snake_case`. No spaces, leading digit or hyphen*.

```
5 + 3  # 8        5 - 3  # 2
5 * 3  # 15       5 / 2  # 2.5
5 // 2 # 2        5 % 2  # 1
2 ** 3 # 8        x += 1 # x = x + 1
```

Order: parentheses → exponents → `* / // %` → `+ -`. **Floor division*** rounds down: `-5 // 2 == -3`.

`import random`: `random()` gives [0,1); `randrange(a,b)` excludes b; `randint(a,b)` includes b. Access with `random.method()`. `seed(n)` makes experiments repeatable.

Week 2 Variables Types Operators

```
name = input("Name: ")  # always str
n = int(input("Number: "))
print("Hello", name)    # space between
print("A", end="")      # no final newline
print(f"{n:.2f}")       # 2 decimal places
```

f-string: `f"{expression:format}"`; `{x=}` shows name/value. Formats: `d` integer, `b` binary, `x/X` hex, `e` exponent, `03d` zero-padding.

Collections and notebook patterns

list `[1,2]`: ordered, mutable, index access.
tuple `(1,2)`: ordered, immutable; singleton `(1,)`.
range `range(start,stop,step)`: stop excluded.
set `{1,2}`: unique, unordered, mutable, no index; empty `set()`.
dict `{"a":1}`: mutable key-value mapping; empty `{}`. Keys must be hashable; insertion order is preserved*.

```
a.append(x)       # add one item
a.extend(items)   # add many
a.pop(i)          # remove/return at index
a.remove(x)       # remove matching value
a.sort()          # in place; returns None*
sorted(a)         # new sorted list
[2*x for x in a]  # list comprehension
```

List: `len, min, max, sum`; `index, count`. `a[::2]` selects even indices. **Set:** `add, remove, update`; `union, intersection, difference`; `pop()` removes an arbitrary element*. **Dict:** `d[k]=v; del d[k]`; `keys(), values(), items(), copy(), update()`. Membership tests keys.

Strings and string manipulation

Strings are immutable. First index = 0; last = -1. `s[start:stop:step]` excludes stop; `s[::-1]` reverses. `+` concatenates; `*` repeats; `in` tests substring membership. `\n` newline, `\t` tab; `ord/chr` convert code points.

`lower, upper, swapcase`; `strip` trims edges*; `replace` returns new text; `split/splitlines` → lists; `sep.join(parts)` → string. `count, startswith, endswith`; `find` returns -1 if missing*, `index` raises an error*. `isalpha/isdigit` classify.

Week 3 Selection Structure

Boolean expression: evaluates True/False. A **branch** runs when its condition is met. Comparisons: `== != < > <= >=`. `=` assigns; `==` compares values; `is` compares identity.

```
if grade >= 80:
    letter = "A"
elif grade >= 70:
    letter = "B"
else:
    letter = "Below B"
```

Single outcome: if; **dual outcome:** if/else; **multiple outcomes:** if/elif/else. First true branch runs. B above means `70 <= grade < 80`. Nested if adds a decision inside a branch. Use colons and consistent indentation*.

and: both true; **or**: at least one true; **not**: reverse Boolean. Order: comparisons → not → and → or. **Decision table:** n Boolean conditions give `2**n` combinations. Conditional expression: `"Pass" if grade >= 50 else "Fail"`.

Notebook Basic I O Methods

```
with open("file.txt") as f:
    text = f.read()
with open("out.txt", "w") as f:
    f.write(text)
```

`read()` → string; `read(n)` → up to n characters; `readlines()` / `list(f)` → lines. Iterate `for line in f`. `writelines` adds no newlines*. `with` closes files*; `"w"` overwrites*. Catch missing files with `except FileNotFoundError:`.

CSV: `csv.reader(f)` → list rows, includes header; `csv.DictReader(f)` → dictionary rows. Convert numeric fields: `int(row["price"]) > 4000`. Write with `csv.writer` / `DictWriter`, `writerow/ writerows`; use `newline=""`. **JSON:** `json.load(f)` reads; `json.dump(data,f)` writes. Navigate nested lists/dicts: `data["results"][0]["name"]`.

Exam reminders and common mistakes*

Trace value **and type** after every step. Test below / at / above boundaries. `"2"+"3"` is `"23"`. `b=a` aliases a collection; `copy()` is shallow. Do not assign mutation results: `a=a.sort()` makes a None. Distinguish value removal from index removal, even values from even indices, and increasing **by** 5 from setting **to** 5.

Source conflicts* The loan prose uses credit >600 for low income; the displayed code uses >=600. Pass/fail prose says above 50; code uses >=50. Follow the version specified. Python allows space indentation; dicts preserve insertion order; set pop is arbitrary.

Sources W1 Introduction to Programming pp. 4-73; W2 Variables Types Operators pp. 2-81; W3 Selection Structure pp. 2-36. Notebooks: variables_data_types Parts 1 and 2; strings Review & Exercise Questions; lists_review; dictionaries_review; IO_methods.

Condensed from the nine-page source-referenced sheet. Examples are adapted from the uploads; starred content is added clarification/study guidance. Full source references and additional corrections are in the comprehensive version.